

P1 — la portion doit dire ce qu'elle représente

Force Tracker — 09/09/2026 — **réponse de conception**, aucune ligne de code écrite. Fait suite à la relecture de ft-v1183 : trois exigences non satisfaites, cinq questions à trancher avant modification.

EN UNE PHRASE

Tes points 1 et 2 sont fondés, et **mon argument tombe** : j'avais refusé `portionWeightG` en écrivant « *cet état n'existe pas* ». C'était vrai du **comportement actuel**, pas du **besoin** — et ton cas « 2 steaks de 125 g » montre que basculer en grammes **détruit l'information « 2 steaks »**. C'est une perte, pas une simplification. Ton point 3, en revanche, est tranché par une mesure et pas par une opinion : **dériver la référence est plus sûr que la stocker** — stocker un entier arrondi perd 1 kcal dès le premier cycle.

CE QUE CE DOCUMENT EST, ET CE QU'IL N'EST PAS

C'est une **proposition de conception**, à valider avant toute modification — la consigne de Michel était explicite : « *ne code pas tout de suite* ». Rien n'a été modifié dans l'application. Le seul commit associé ajoute la mesure d'arrondi au journal de test, pour qu'elle ne disparaisse pas avec la session. Tout ce qui est chiffré ici a été **exécuté dans un vrai navigateur** (Chromium + Playwright), jamais déduit du code.

0. Ce que je concède, et pourquoi mon argument était faux

Mon refus reposait sur une mesure exacte : déclarer un poids fait aujourd'hui basculer l'application en grammes, donc l'état « portion dont on connaît le poids » n'existe pas, donc le champ ne se remplirait jamais. **Chaque maillon est vrai, et la conclusion est fausse** — parce que j'ai mesuré ce que l'application *fait* et conclu sur ce qu'elle *doit* faire.

LA FORME EXACTE DE L'ERREUR, PARCE QU'ELLE SE REPRODUIRA

Une mesure du **comportement actuel** ne peut jamais justifier un refus de besoin. Elle décrit ce qui est, pas ce qui manque. Le seul refus qu'elle autorise est « *ce champ est aujourd'hui inutile* » — jamais « *ce champ ne servira jamais* ». Le projet a déjà une règle pour ça (une limite affirmée sans être vérifiée devient une règle de conception silencieuse) ; ici la limite **était** vérifiée, et c'est la question posée qui était mauvaise.

1. Comment stocker `portionLabel`

Un champ sur l'entrée du journal : chaîne courte, **écrite par la personne, jamais devinée**.

Décision	Contenu	Pourquoi
On stocke le nom nu	"steak", pas "1 steak (125 g)"	L'écran compose la phrase. Stocker la phrase figerait une formulation dans la donnée, et toute évolution d'affichage deviendrait une migration.
Singulier, décrit UNE portion	"steak" -> l'écran rend « 2 portions — 1 steak »	Le pluriel est un problème d'affichage, pas de donnée.
Absent -> null	L'écran dit « 1 portion (non définie) »	On n'invente pas un nom. Un manque nommé vaut mieux qu'un manque comblé.
Longueur bornée	24 caractères	C'est une étiquette, pas une note. Au-delà, l'écran ne peut plus la composer.

Où se saisit-il ?

Un champ court dans le bloc portions, **plus une rangée de puces** (steak · part · yaourt · dose · sachet · tranche · bol · assiette). **La rangée de puces n'est pas du confort** : taper une étiquette au clavier à chaque repas sur un téléphone ne tiendrait pas trois jours, et un champ qu'on ne remplit plus est pire qu'un champ absent — il donne l'illusion que l'information existe.

CE QUI REND LA FONCTION SUPPORTABLE À L'USAGE — ET RIEN N'EST À CONSTRUIRE

« Mes aliments » et les favoris portent **déjà** q, u et per100 d'une saisie à l'autre. Le label et le poids empruntent **exactement le même chemin** : reprendre « Steak haché » repropose « 1 steak, 125 g » sans rien retaper. **L'étiquette ne se saisit donc qu'une fois par aliment**, pas une fois par repas.

LE PIÈGE À NOMMER : LE FAVORI GARDE UNE COPIE

C'est déjà vrai de per100, et ça a déjà coûté dans ce projet — une copie fausse survit à la correction de la ligne d'origine, et personne ne voit d'où elle vient. **La vérité doit rester sur l'entrée du journal ; la copie du favori n'est qu'un pré-remplissage.** Corriger une étiquette sur une entrée ne doit pas laisser l'ancienne vivre indéfiniment dans les favoris — c'est un point à trancher explicitement, pas à découvrir dans six mois.

2. Conserver portionWeightG sans casser les grammes

Le nœud est là : aujourd'hui, déclarer un poids **change l'unité**. Il faut découpler **ce qu'on compte de la conversion en masse**. Deux questions différentes, deux champs, un propriétaire chacun :

La question posée	Le champ	Exemple « 2 steaks »
Combien j'en ai mangé ?	q + u	q:2, u: 'portion'
Combien pèse UNE portion ?	portionWeightG	125
Combien ça pèse en tout ?	dérivé : q x portionWeightG	250 g — jamais stocké
Combien pour 100 g ?	dérivable : totaux / (q x poids) x 100	voir l'arbitrage ci-dessous

Ce qui change, et surtout ce qui ne change pas

L'onglet « en grammes » garde son rôle à l'identique : on compte en grammes, u : 'g', le poids de portion n'est pas pertinent. C'est **ce qui protège les quatre corrections précédentes** — toute la famille de bugs de contamination de quantité vit sur ce chemin, et il ne doit pas bouger.

L'onglet « en portions » gagne un champ optionnel « une portion pèse ... g ». Le renseigner **ne bascule plus l'unité** : u reste 'portion', q reste 2. C'est littéralement ta demande n°2.

LE PIÈGE STRUCTUREL, ET IL EST SÉRIEUX

La variable qui porte aujourd'hui le poids déclaré signifie « *poids de ce qui est affiché* », donc du **total**. Dans l'onglet portions, le nouveau champ signifie « *poids d'UNE portion* ». **Deux notions, deux nombres, et ils ne doivent jamais partager une variable.** Le catalogue de bugs du projet a une famille entière pour ça (« deux sources qui se contredisent ») et une autre pour les quantités qui fuient d'un aliment au suivant. Concrètement : noms distincts, remise à zéro sur les mêmes chemins, et **un témoin sur chacun**.

L'arbitrage que je ne tranche pas seul

Quand portionWeightG est connu, per100 devient calculable. Faut-il **l'écrire** dans la donnée ?

Option	Ce qu'on y gagne	Ce qu'on y perd
(a) L'écrire, recalculé à chaque écriture	Tout le code existant en profite sans modification : les reprises, l'écran d'édition, le champ grammes, le garde-fou de masse. Aucun chantier ouvert dans les lecteurs.	Une redondance : deux chemins mènent au même nombre, et ils peuvent diverger si on corrige le poids de portion plus tard sans recalculer.
(b) Ne pas l'écrire, adapter les lecteurs	Une seule source de vérité, strictement.	Touche beaucoup de lecteurs, donc beaucoup de surface de régression sur ce qui vient juste d'être stabilisé.

Ma recommandation : (a), avec une règle écrite — *per100 est recalculé à chaque écriture, jamais conservé tel quel*. C'est le choix qui n'ouvre pas de chantier dans du code qu'on vient de corriger quatre fois. Mais c'est un arbitrage de conception, pas une évidence technique : **il se tranche, il ne se déduit pas**.

3. La référence d'une portion : dérivée ou stockée ? (mesuré)

Ton exemple exact — **601 kcal pour 3 portions** — joué dans un vrai navigateur, **dix cycles** de reprise puis réenregistrement, sans jamais rien retoucher :

Stratégie	Référence d'une portion	Totaux après 10 cycles
Dérivée (totaux / q) — le code actuel	200,3333 — stable	601 — inchangés
Stockée arrondie	200	600 dès le 1er cycle, puis figé

LA RAISON DE FOND, ET ELLE N'EST PAS L'ÉLÉGANCE

Les **totaux** sont la donnée que la personne a validée à l'écran ; q est son choix. **La référence n'est qu'une VUE de ces deux-là**. Stocker une vue crée une seconde source, qu'il faut arrondir pour l'écrire — et c'est l'arrondi qui perd la kcal, pas la division.

LA LIMITE DE CETTE CONCLUSION, ÉCRITE POUR QU'ON NE LA SUR-GÉNÉRALISE PAS

Elle ne tient **que** parce que les totaux sont la vérité stockée. Si on décidait que la **référence** est la vérité et les totaux la vue — ce qui est parfaitement défendable, et c'est d'ailleurs la forme de ta spécification d'origine — **la conclusion s'inverserait**. Ce n'est pas le modèle actuel, et en changer toucherait tous les lecteurs. *Si tu veux ce basculement, c'est un chantier à part entière, pas un champ de plus.*

ET MA PREMIÈRE MESURE ÉTAIT FAUSSE — je le dis parce que c'est le genre d'erreur qui rassure

J'avais saisi 601 kcal **par portion** (donc 1803 au total). La division tombait juste, et le tableau montrait « aucune dérive » — **alors qu'il ne testait pas la fraction du tout**. Corrigé en posant une ligne qui porte 601 **au total** pour q : 3. *Une mesure qui n'exerce pas le cas qu'on croit mesurer ne prouve rien, elle rassure.*

4. Les lecteurs et écrivains à adapter

Issu de l'audit des huit chemins fait pour ft-v1183, mis à jour pour les deux nouveaux champs.

Rôle	Fonctions	Note
Écrivains	<code>_provFood · _afSetSrc · quickAddFood · saveEditFood · rejouerRepas · toggleFavFood</code>	La liste blanche de _provFood est le point noir : c'est la QUATRIÈME fois qu'un champ posé en amont n'y est pas recopié et n'atteint jamais la donnée, sans erreur ni test rouge. Elle mérite un témoin dédié, pas seulement de l'attention.
Lecteurs	<code>quickFillFood</code> et <code>_afSuggPrendreLocale · _afMajAncre · _efQtyRender · _renderFoodQuickList</code>	Les deux portes de reprise se corrigent toujours ensemble : un mécanisme posé sur une seule des deux est le motif recensé sept fois dans ce fichier.
Export CSV	<code>exportNutritionCsv (setup.js)</code>	Deux colonnes à ajouter (<code>portion_label</code> , <code>portion_poids_g</code>). Sans elles, l'export perd précisément l'information qu'on vient de sauver.
Rien à faire	<code>Synchro cloud · Code.js</code>	Passe-plats, mesuré : les objets du journal traversent sans filtre ni transformation.
Non touchés	<code>_qtyRescale · recherche / CIQUAL · scan code-barres · migration</code>	Explicitement hors périmètre, comme pour ft-v1183.

DEUX OPPORTUNITÉS QUE JE SIGNALE SANS LES PRENDRE

(1) Le garde-fou de masse (« ces calories ne peuvent pas tenir dans ce poids ») **se tait aujourd'hui en portions**, faute de masse connue. Avec `portionWeightG`, il pourrait enfin parler. (2) Le journal du jour **n'affiche aucune quantité** — ni grammes ni portions. C'est pourtant l'écran où « 2 steaks » se lirait vraiment. *Les deux sont hors de ce qui a été demandé. Je les nomme pour qu'elles soient des décisions et non des oublis.*

5. Les tests qui empêcheront une régression

Ce qui doit rester vert (5 blocs de témoins existants)

Bloc	Ce qu'il fige	Valeurs interdites
CCLXXVII (ft-v1179)	Un bloc pour-100 g visible sans source ne donne pas sa quantité	100 · 160
CCLXXVIII (ft-v1180)	Les cinq contaminations d'un aliment au suivant	380 · 150 · 200
CLXVIII (ft-v1181)	Le couple référence / quantité survit à l'aller-retour d'onglet	117/26 · 156/35 · q=30
CCLXXIX (ft-v1182)	Les résultats de recherche restent visibles à l'écran	—
CCLXXX (ft-v1183)	Les 15 témoins de la portion, dont « rien n'est écrit sans choix »	q non nul sans geste

Ce qu'il faut ajouter (10 témoins)

(1) « 2 steaks de 125 g » -> q:2, u: 'portion', étiquette et poids persistés, masse totale 250 g. (2) L'écran rend « 1 portion = 1 steak (125 g) », et **jamais** un poids quand il est inconnu. (3) **Déclarer le poids d'une portion ne bascule pas l'unité** — c'est le témoin qui garde ta demande n°2, et le plus important des dix. (4) L'onglet grammes est **inchangé** : 500 g reste q:500, u: 'g'. (5) Reprendre une ligne « 2 steaks » repropose tout, sans rien retaper. (6) D'un aliment à l'autre, **ni l'étiquette ni le poids de portion ne traversent**. (7) Étiquette absente -> « portion (non définie) », aucune invention. (8) 601 kcal / 3 portions -> référence stable et totaux à 601 sur dix cycles. (9) L'export CSV porte les deux nouvelles colonnes. (10) Le pour-100 g dérivé est cohérent avec les totaux.

Les contrôles négatifs prévus

Chaque protection doit pouvoir **rougir** : retirer l'étiquette de la liste blanche · retirer le poids de portion · faire rebasculer l'unité quand on déclare le poids · stocker une référence arrondie · faire traverser l'étiquette d'un aliment au suivant.

ET LA RÈGLE QUE CE CHANTIER DOIT S'APPLIQUER À LUI-MÊME

Sur la version précédente, **deux mutations n'ont fait rougir personne** — et ce n'était pas du code mort, c'étaient des **trous de témoin**. Sans les combler, le contrôle négatif aurait déclaré mortes deux vraies protections, et je les aurais retirées. **Toute protection ajoutée ici devra être éprouvée par sa propre mutation avant d'être considérée comme acquise.**

6. Ce qui reste à trancher avant que je code

Décision	Options	Ma recommandation
Écrire per100 quand il devient calculable ?	(a) l'écrire, recalculé à chaque écriture · (b) ne pas l'écrire et adapter les lecteurs	(a) — n'ouvre pas de chantier dans du code corrigé quatre fois
L'étiquette du favori quand on corrige celle de l'entrée	la mettre à jour · la laisser vivre comme simple pré-remplissage	Pré-remplissage , et l'écrire — sinon on redécouvre le problème dans six mois
Le journal du jour doit-il afficher « 2 steaks » ?	oui · non, chantier séparé	Chantier séparé : c'est un écran, pas une donnée

Décision	Options	Ma recommandation
Le garde-fou de masse doit-il parler en portions ?	oui, dès que le poids est connu · non	Oui , mais après — il ne conditionne pas la structure

OÙ LIRE LA SUITE

Le débrief de la version livrée est dans docs/P1-PORTIONS-DEBRIEF . pdf. La mesure d'arrondi complète, avec l'erreur de fixture, est consignée dans docs/JOURNAL-DE-TEST . md. Le journal de version est dans CLAUDE . md (entrée ft-v1183) et l'état courant dans docs/CONTEXTE-ACTUEL . md. Le dépôt est public : github.com/michdu75-commits/forcetracker.